CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 29: VAE/GAN Sample Generation

Problem Statement: A fashion retailer wants to generate new product images to prototype designs before manufacturing. The Fashion-MNIST dataset (clothing and footwear images, 28×28). The objective is to build a VAE and a GAN and generate new samples. Use TensorFlow, Keras, NumPy, Matplotlib.

Tasks:

1. Build a VAE with encoder, reparameterisation and decoder; train with reconstruction plus KL loss.

2. Generate images by sampling the latent space; plot a 2-D latent-space grid.

3. Build a DCGAN with a generator and a discriminator.

4. Train the GAN and save generated images every few epochs.

5. Plot generator and discriminator loss curves.

FORMULA:

1. Encoder Mean

$\mu = W\mu x + b\mu$

2. Encoder Log Variance

$\log(\sigma^2) = W\sigma x + b\sigma$

3. Reparameterisation Trick

$$z = \mu + \sigma \times \varepsilon$$

where $\varepsilon \sim N(0, I)$

4. Gaussian Sampling

$$\varepsilon \sim N(0, I)$$

5. Decoder Output

$$\hat{x} = \text{Decoder}(z)$$

6. VAE Reconstruction Loss

$$L_{recon} = -\Sigma_i [x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i)]$$

7. KL Divergence Loss

$$L_{KL} = -\frac{1}{2} \times \Sigma_i [1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2]$$

8. Total VAE Loss

$$L(VAE) = L_{recon} + L_{KL}$$

9. Latent Space Sampling

$$z \sim N(0, I)$$

10. Generator Function

$$x_{fake} = G(z)$$

11. Discriminator Function

$D(x)$ = Probability that x is real

12. GAN Discriminator Loss

$$L_D = -[\log D(x_{real}) + \log(1 - D(G(z)))]$$

13. GAN Generator Loss

$LG = -\log D(G(z))$

14. GAN Min-Max Objective

$\min G \max D\ V(D,G) = E_x[\log D(x)] + E_z[\log(1 - D(G(z)))]$

15. Binary Cross-Entropy Loss

$BCE = -(1/N) \times \Sigma_i [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$

16. Generator and Discriminator Loss

Total GAN Loss = LG + LD

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

np.random.seed(42)

tf.random.set_seed(42)

# Load Fashion-MNIST

(x_train, _), (_, _) = keras.datasets.fashion_mnist.load_data()

x_train = x_train[:12000].astype("float32") / 255.0

x_train = np.expand_dims(x_train, -1)
```

```
# VAE Encoder

latent_dim = 2

encoder_input = keras.Input(shape=(28, 28, 1))

x = layers.Flatten()(encoder_input)

x = layers.Dense(256, activation="relu")(x)

mean = layers.Dense(latent_dim)(x)

log_var = layers.Dense(latent_dim)(x)

encoder = keras.Model(

    encoder_input, [mean, log_var]

)



# VAE Decoder

decoder_input = keras.Input(shape=(latent_dim,))

x = layers.Dense(256, activation="relu")(decoder_input)

x = layers.Dense(784, activation="sigmoid")(x)

decoder_output = layers.Reshape((28, 28, 1))(x)

decoder = keras.Model(

    decoder_input, decoder_output

)

# VAE Model
```

```
class VAE(keras.Model):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.total_loss_tracker = keras.metrics.Mean(name="loss")
        self.reconstruction_tracker = keras.metrics.Mean(name="reconstruction_loss")
        self.kl_tracker = keras.metrics.Mean(name="kl_loss")
    @property
    def metrics(self):
        return [
            self.total_loss_tracker,
            self.reconstruction_tracker,
            self.kl_tracker
        ]
    def train_step(self, data):
        images = data[0] if isinstance(data, tuple) else data
        with tf.GradientTape() as tape:
            mean, log_var = self.encoder(images)
            epsilon = tf.random.normal(tf.shape(mean))
```

```
    z = mean + tf.exp(0.5 * log_var) * epsilon
    reconstructed = self.decoder(z)
    reconstruction_loss = tf.reduce_mean(
        tf.reduce_sum(
            keras.losses.binary_crossentropy(
                images, reconstructed
            ), axis=(1, 2)
        )
    )
    kl_loss = -0.5 * tf.reduce_mean(
        tf.reduce_sum(
            1 + log_var - tf.square(mean) -
            tf.exp(log_var), axis=1
        )
    )
    total_loss = reconstruction_loss + kl_loss
gradients = tape.gradient(
    total_loss, self.trainable_weights
)
self.optimizer.apply_gradients(
```

```
            zip(gradients, self.trainable_weights)
        )
        self.total_loss_tracker.update_state(total_loss)
        self.reconstruction_tracker.update_state(reconstruction_loss)
        self.kl_tracker.update_state(kl_loss)
        return {
            "loss": self.total_loss_tracker.result(),
            "reconstruction_loss": self.reconstruction_tracker.result(),
            "kl_loss": self.kl_tracker.result()
        }


vae = VAE(encoder, decoder)
vae.compile(optimizer=keras.optimizers.Adam())
print("Training VAE")
vae_history = vae.fit(
    x_train,
    epochs=10,
    batch_size=128
)
```

```
# Generate VAE samples
z = np.random.normal(size=(10, latent_dim)).astype("float32")
generated = decoder.predict(z, verbose=0)
plt.figure(figsize=(12, 3))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(generated[i].squeeze(), cmap="gray")
    plt.axis("off")
plt.suptitle("VAE Generated Images")
plt.show()


# Plot 2D latent space grid
grid = np.linspace(-3, 3, 10)
latent_points = np.array(
    [[x, y] for y in grid for x in grid],
    dtype="float32"
)
images = decoder.predict(latent_points, verbose=0)
plt.figure(figsize=(10, 10))
for i in range(100):
```

```
    plt.subplot(10, 10, i + 1)
    plt.imshow(images[i].squeeze(), cmap="gray")
    plt.axis("off")
plt.suptitle("VAE 2D Latent Space Grid")
plt.show()
# Prepare GAN data
gan_data = (x_train * 2.0) - 1.0
# GAN Generator
noise_dim = 100
generator = keras.Sequential([
    layers.Input(shape=(noise_dim,)),
    layers.Dense(7 * 7 * 128, use_bias=False),
    layers.BatchNormalization(),
    layers.LeakyReLU(negative_slope=0.2),
    layers.Reshape((7, 7, 128)),
    layers.Conv2DTranspose(
        64, 4, strides=2, padding="same", use_bias=False
    ),
    layers.BatchNormalization(),
    layers.LeakyReLU(negative_slope=0.2),
```

```
    layers.Conv2DTranspose(
        1, 4, strides=2, padding="same", activation="tanh"
    )
])


# GAN Discriminator
discriminator = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(64, 4, strides=2, padding="same"),
    layers.LeakyReLU(negative_slope=0.2),
    layers.Dropout(0.3),
    layers.Conv2D(128, 4, strides=2, padding="same"),
    layers.LeakyReLU(negative_slope=0.2),
    layers.Dropout(0.3),
    layers.Flatten(),
    layers.Dense(1)
])
# GAN Optimizers
g_optimizer = keras.optimizers.Adam(0.0002, beta_1=0.5)
d_optimizer = keras.optimizers.Adam(0.0002, beta_1=0.5)
```

```
loss_function = keras.losses.BinaryCrossentropy(
    from_logits=True
)
# GAN Loss Functions
def generator_loss(fake_output):
    return loss_function(
        tf.ones_like(fake_output), fake_output
    )
def discriminator_loss(real_output, fake_output):
    real_loss = loss_function(
        tf.ones_like(real_output), real_output
    )
    fake_loss = loss_function(
        tf.zeros_like(fake_output), fake_output
    )
    return real_loss + fake_loss
# GAN Training
dataset = tf.data.Dataset.from_tensor_slices(gan_data)
dataset = dataset.shuffle(12000).batch(128)
@tf.function
```

```
def train_step(real_images):
    noise = tf.random.normal((tf.shape(real_images)[0], noise_dim))
    with tf.GradientTape() as gt, tf.GradientTape() as dt:
        fake_images = generator(noise, training=True)
        real_output = discriminator(real_images, training=True)
        fake_output = discriminator(fake_images, training=True)
        g_loss = generator_loss(fake_output)
        d_loss = discriminator_loss(real_output, fake_output)
    g_gradients = gt.gradient(
        g_loss, generator.trainable_variables
    )

    d_gradients = dt.gradient(
        d_loss, discriminator.trainable_variables
    )
    g_optimizer.apply_gradients(
        zip(g_gradients, generator.trainable_variables)
    )
    d_optimizer.apply_gradients(
        zip(d_gradients, discriminator.trainable_variables)
```

```
    )
    return g_loss, d_loss
# Save generated images
def save_images(epoch):
    noise = tf.random.normal((16, noise_dim))
    images = generator(noise, training=False)
    images = (images + 1) / 2
    plt.figure(figsize=(6, 6))
    for i in range(16):
        plt.subplot(4, 4, i + 1)
        plt.imshow(images[i, :, :, 0], cmap="gray")
        plt.axis("off")
    plt.suptitle("GAN Epoch " + str(epoch))
    plt.tight_layout()
    plt.show()
# Train GAN
g_losses = []
d_losses = []
print("Training GAN")
for epoch in range(10):
```

```
    total_g = 0
    total_d = 0
    batches = 0
    for real_images in dataset:
        g_loss, d_loss = train_step(real_images)
        total_g += float(g_loss)
        total_d += float(d_loss)
        batches += 1
    g_losses.append(total_g / batches)
    d_losses.append(total_d / batches)


    print(
        "Epoch:", epoch + 1,
        "Generator Loss:", round(g_losses[-1], 4),
        "Discriminator Loss:", round(d_losses[-1], 4)
    )
    if (epoch + 1) % 5 == 0:
        save_images(epoch + 1)
# Plot GAN losses
plt.figure(figsize=(8, 5))
```

```
plt.plot(g_losses, label="Generator Loss")

plt.plot(d_losses, label="Discriminator Loss")

plt.title("GAN Generator and Discriminator Loss")

plt.xlabel("Epochs")

plt.ylabel("Loss")

plt.legend()

plt.grid()

plt.show()
```

OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz

29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz

26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz

5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz

4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Training VAE
```

Epoch 1/10

94/94 ━━━━━━━━━━━━━━━━━━━━ **6s** 29ms/step - kl_loss: 16.3024 - loss: 376.0636 - reconstruction_loss: 359.7612

Epoch 2/10

94/94 ━━━━━━━━━━━━━━━━━━━━ **0s** 3ms/step - kl_loss: 9.5319 - loss: 300.9391 - reconstruction_loss: 291.4072

Epoch 3/10

94/94 ━━━━━━━━━━━━━━━━━━━━ **0s** 3ms/step - kl_loss: 7.7108 - loss: 290.7587 - reconstruction_loss: 283.0479

Epoch 4/10

94/94 ━━━━━━━━━━━━━━━━━━━━ **0s** 3ms/step - kl_loss: 7.2114 - loss: 286.6366 - reconstruction_loss: 279.4252

Epoch 5/10

94/94 ━━━━━━━━━━━━━━━━━━━━ **0s** 3ms/step - kl_loss: 6.9362 - loss: 284.0175 - reconstruction_loss: 277.0812

Epoch 6/10

94/94 ━━━━━━━━━━━━━━━━━━━━ **0s** 3ms/step - kl_loss: 6.7786 - loss: 282.2029 - reconstruction_loss: 275.4244

Epoch 7/10

94/94 ━━━━━━━━━━━━━━━━━━━━ **0s** 3ms/step - kl_loss: 6.6290 - loss: 281.1702 - reconstruction_loss: 274.5412

Epoch 8/10

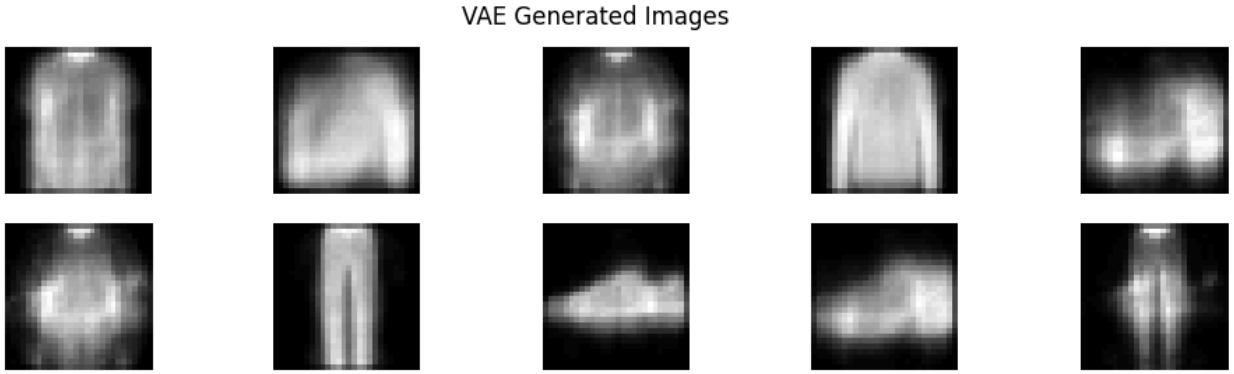
94/94 ━━━━━━━━━━━━━━━━━━━━ **0s** 3ms/step - kl_loss: 6.5466 - loss: 280.0912 - reconstruction_loss: 273.5447

Epoch 9/10

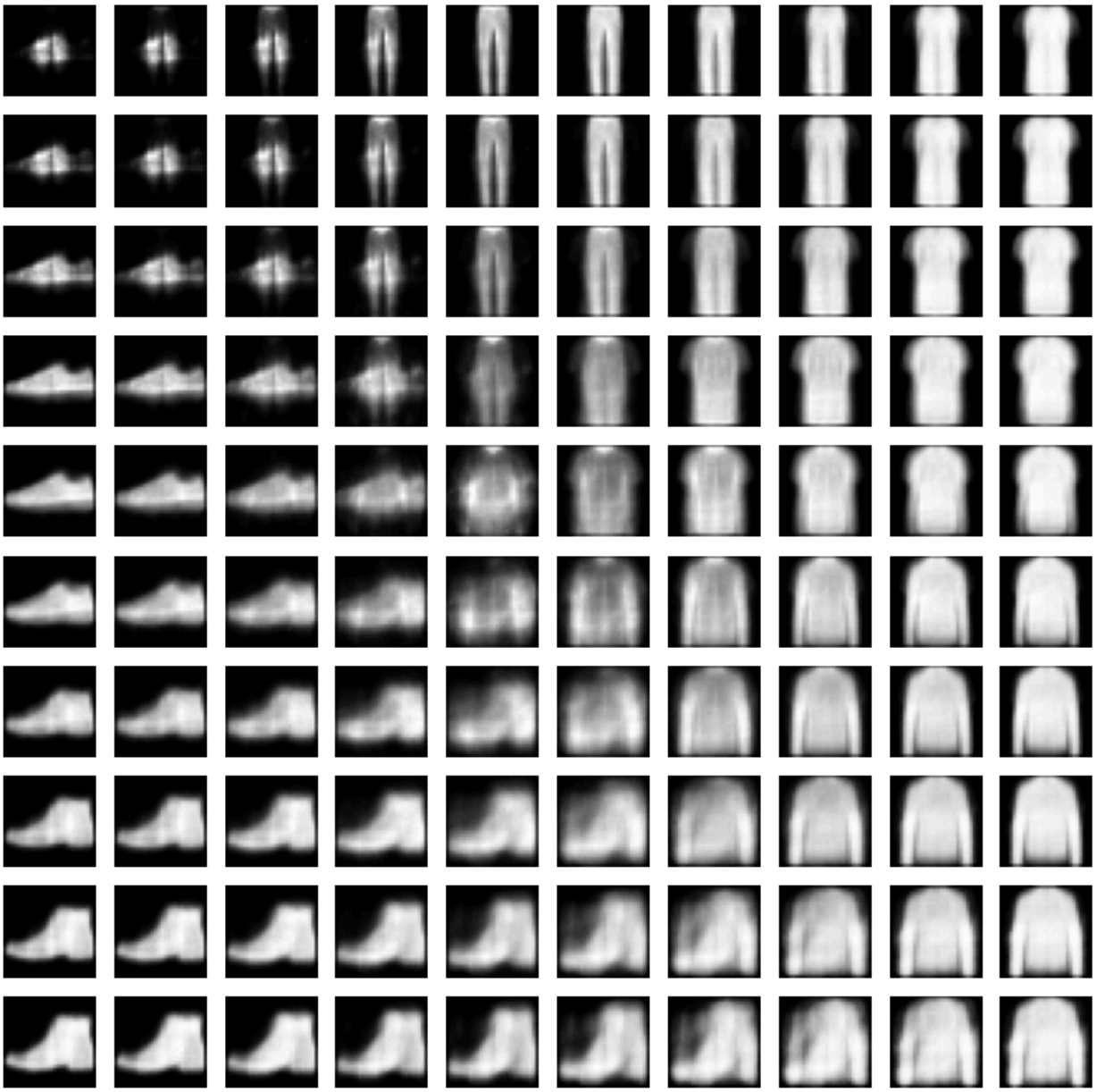
94/94 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 3ms/step - kl_loss: 6.4587 - loss: 279.2672 - reconstruction_loss: 272.8084

Epoch 10/10

94/94 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 3ms/step - kl_loss: 6.4487 - loss: 278.2982 - reconstruction_loss: 271.8494



VAE 2D Latent Space Grid



Training GAN

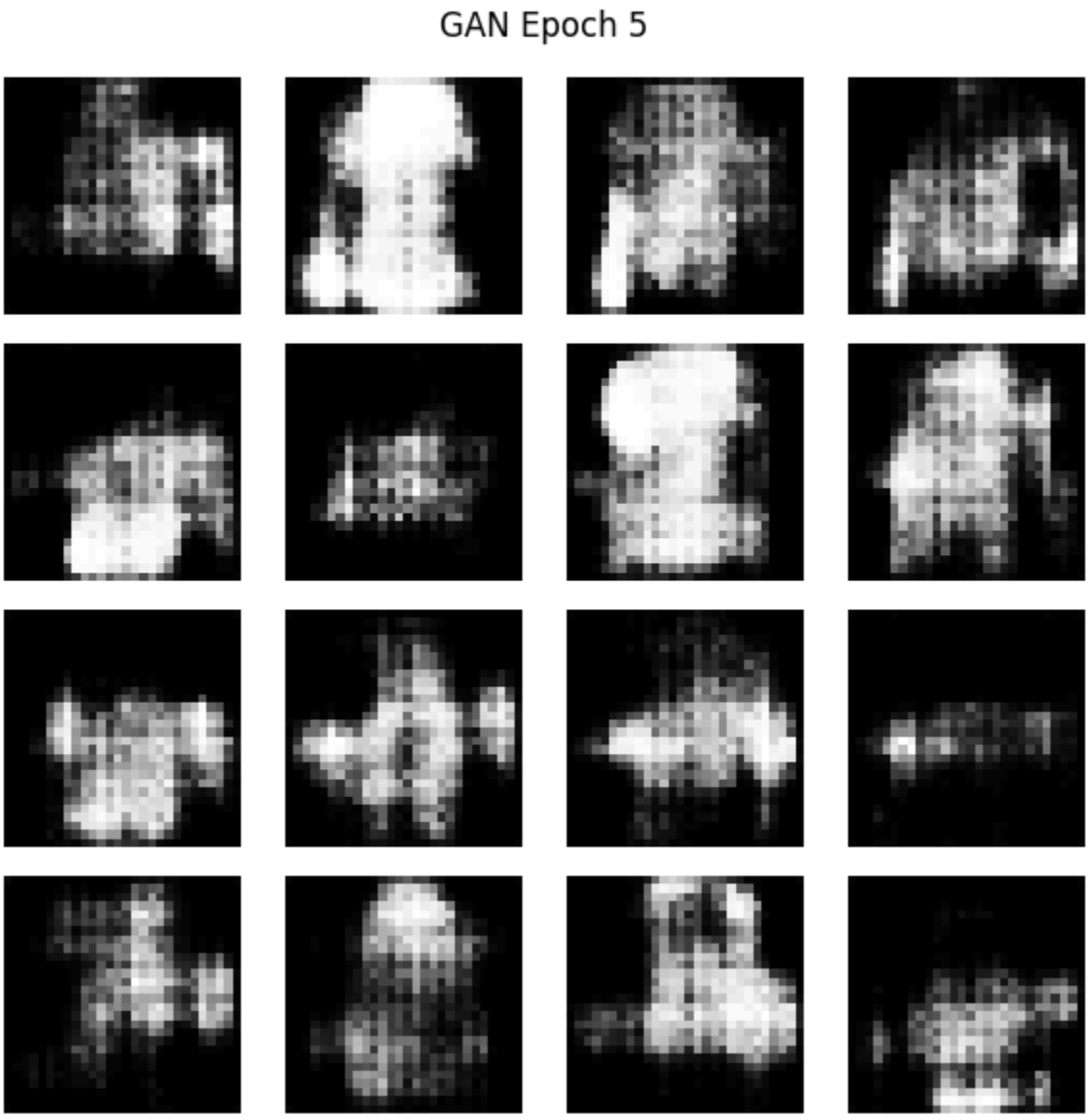
Epoch: 1 Generator Loss: 0.7618 Discriminator Loss: 1.0575

Epoch: 2 Generator Loss: 0.7512 Discriminator Loss: 1.2982

Epoch: 3 Generator Loss: 0.7222 Discriminator Loss: 1.3786

Epoch: 4 Generator Loss: 0.7363 Discriminator Loss: 1.3371

Epoch: 5 Generator Loss: 0.7637 Discriminator Loss: 1.3042



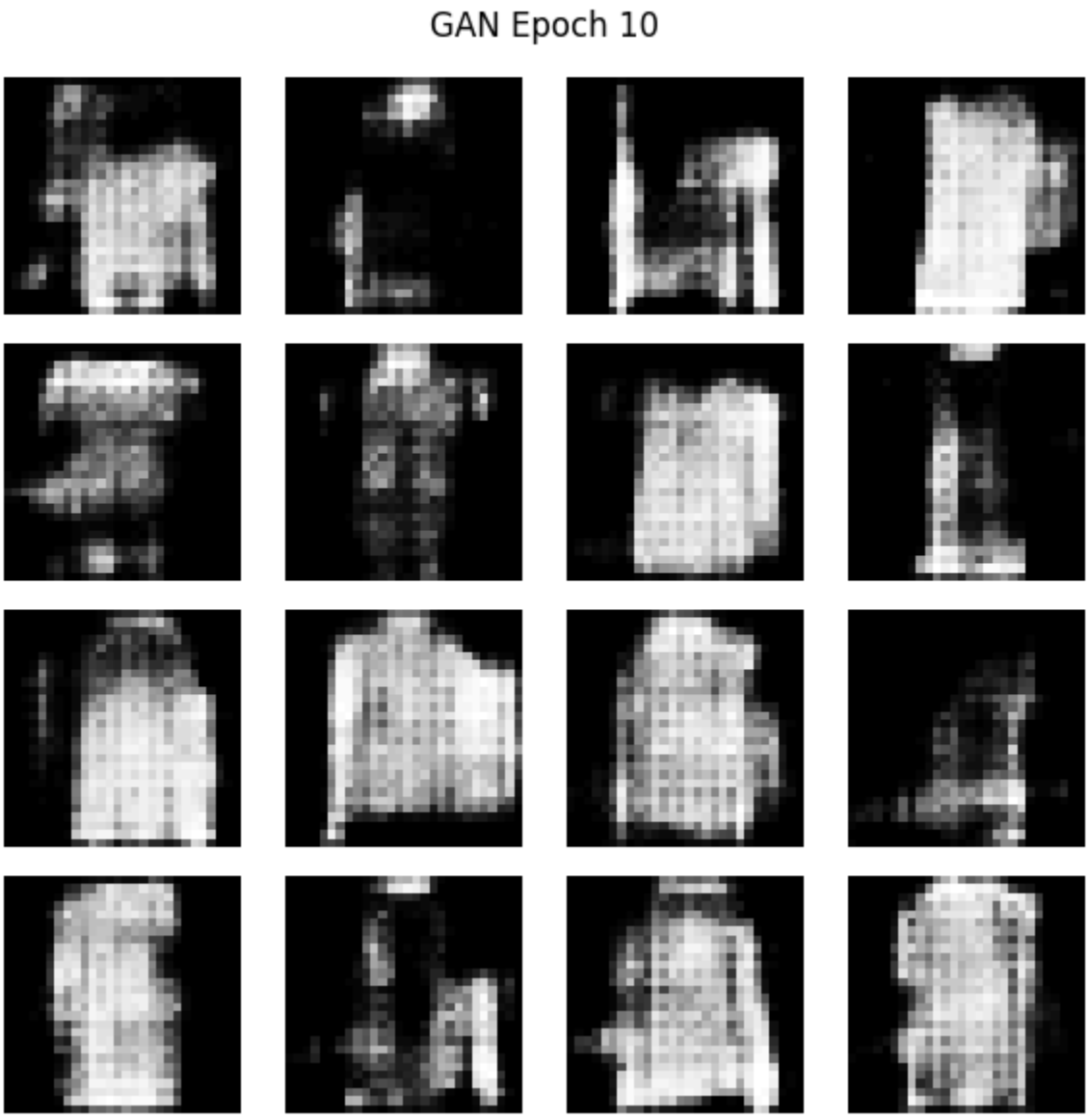
Epoch: 6 Generator Loss: 0.7862 Discriminator Loss: 1.2731

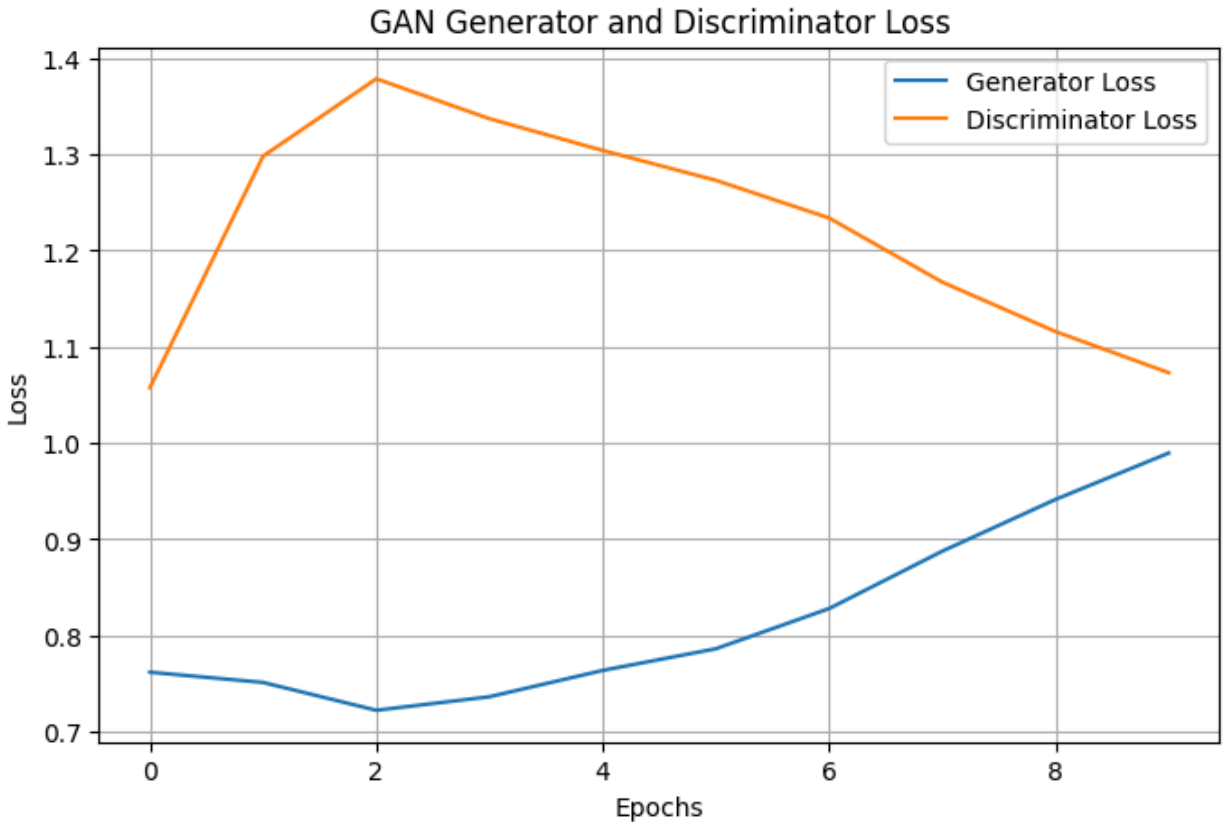
Epoch: 7 Generator Loss: 0.828 Discriminator Loss: 1.234

Epoch: 8 Generator Loss: 0.8876 Discriminator Loss: 1.1673

Epoch: 9 Generator Loss: 0.9414 Discriminator Loss: 1.1159

Epoch: 10 Generator Loss: 0.9896 Discriminator Loss: 1.0731




GAN Generator and Discriminator Loss
Generator Loss
Discriminator Loss
Loss
Epochs
1.4
1.3
1.2
1.1
1.0
0.9
0.8
0.7
0
2
4
6
8